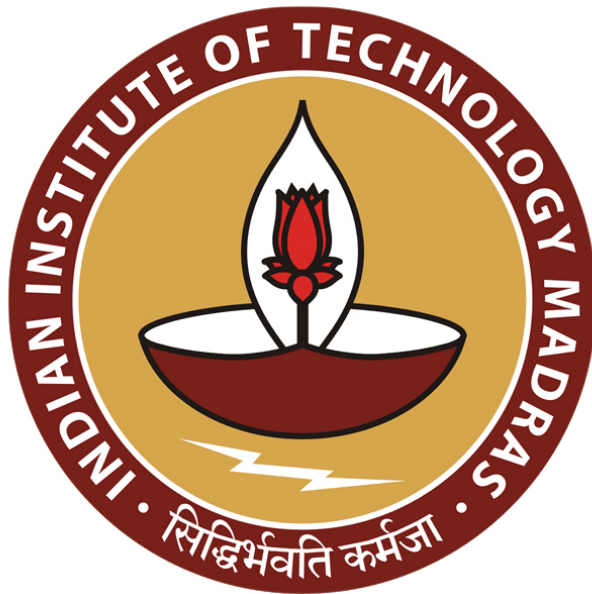


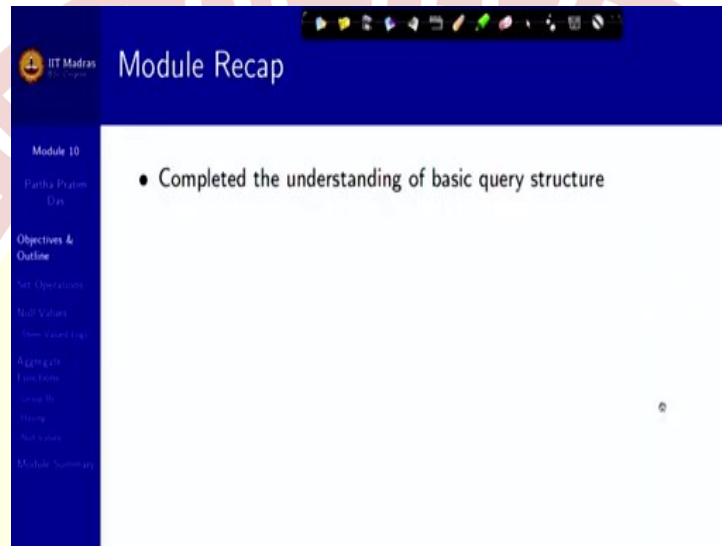


IIT Madras
ONLINE DEGREE

Database Management Systems
Professor Partha Pratim Das
Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur
Introduction to SQL/3

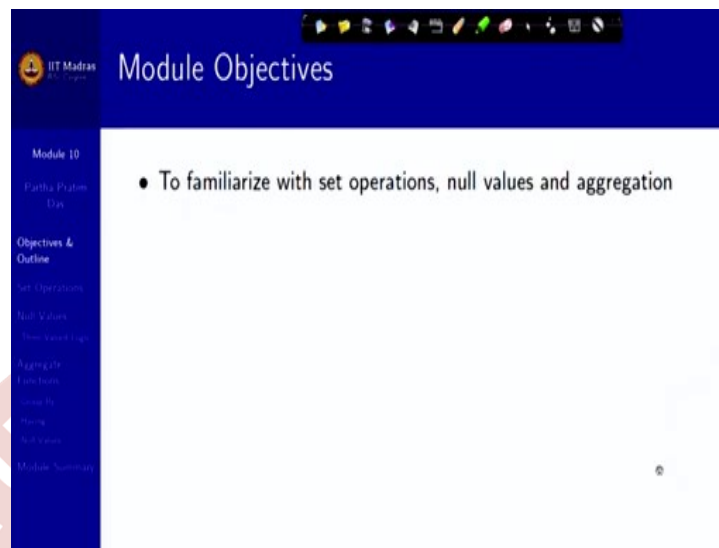
Welcome to module 10 of Database Management Course in the IIT Madras Online B.Sc. Program.

(Refer Slide Time: 0:30)



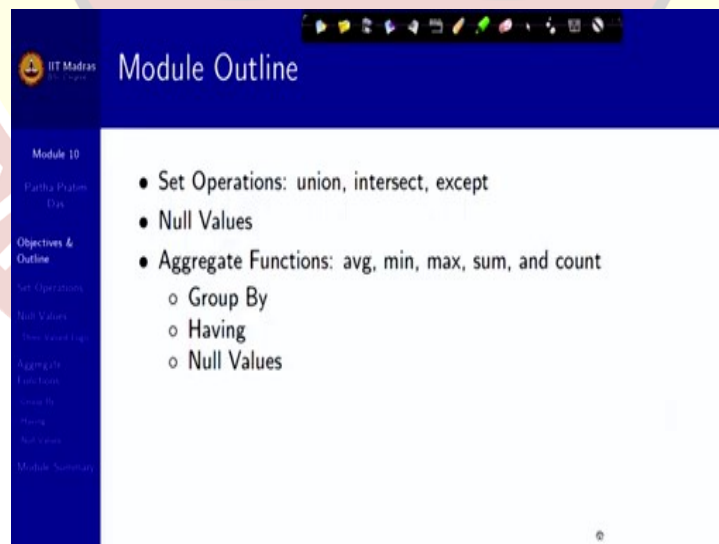
We are in the middle of learning SQL. In module 8 we had discussed about the basic query structure and in the last module we have talked in some more length in different additional operations and may be supporting features and so on.

(Refer Slide Time: 0:55)



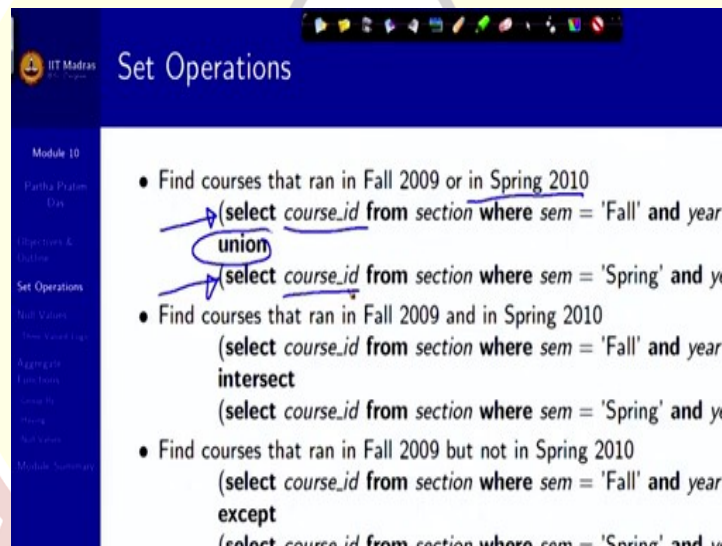
So, continuing on that we will in this module talk about some of the little other ways of doing the operations in SQL, not only being limited to the basic structure, this will include set operations, this will include discussion on how does null values impact the queries and the results and finally, certainly the aggregation functions I had mentioned couple of modules back.

(Refer Slide Time: 1:41)



So these are, this is the outline which you will see on the left.

(Refer Slide Time: 1:48)



Set Operations

- Find courses that ran in Fall 2009 **or** in Spring 2010
`(select course_id from section where sem = 'Fall' and year = 2009)`
union
`(select course_id from section where sem = 'Spring' and year = 2010)`
- Find courses that ran in Fall 2009 **and** in Spring 2010
`(select course_id from section where sem = 'Fall' and year = 2009)`
intersect
`(select course_id from section where sem = 'Spring' and year = 2010)`
- Find courses that ran in Fall 2009 **but not in** Spring 2010
`(select course_id from section where sem = 'Fall' and year = 2009)`
except
`(select course_id from section where sem = 'Spring' and year = 2010)`

Set operations, now the set operations, when we talk about set operations we very simply treat relations as sets. They are no more multisets and they are treated as sets, so two relations can be involved in a binary set operation like union, intersection or difference provided they are identical in the structure. I mean, we cannot take one relation having some three attributes and another relation having some two attributes and do a union; does not make sense.

And more often the set operation is usually done between two different derived forms of the same relation. So, let us see how. So, we want to, we are considering the courses table, courses relation where all course IDs section, semester, year, all those are specified. This is from the same university Schema and we want to find all courses that ran in 2009 or in 2010.

Now, let us look at the first part of this query that ran in 2010, 2009. So, how do you get that? Very simple, you want the courses, so you put select course ID, you want it from the table section and the condition is semester has to be fall, year has to be 2009; that will give you all courses that ran in fall 2009. Now, similarly the second part, all courses that ran in spring 2010 is this.

So, all that you need is the courses that ran either in fall 2009 or in spring 2010, you just need to take a union of them, so you do union. So, this is one table result, this is another table result, they have identical structure, which is a course ID, in this case just a single column and you put a union of all the records.

Now, once you get this idea, then naturally, so basically the or here is getting expressed by this union here, so similarly if I just modify this query and want to know that get me the

courses that ran both in fall 2009 as well as in spring 2010, again I will form the basic queries and I want those course IDs to be on both so I will do an intersect, I will do an intersection.

So and basically here gives you that intersection. Now, if I want to do some exclusion, say I want courses that ran in fall 2009 but not in spring 2010, so the courses ran in for 2009, the are courses in fall 2010 we know which courses ran in both, so we want to take out the courses from fall 2009 which were also offered in spring 2010 and just leave the rest.

So, this is set difference. So, but not in here, is done by the set difference for which the keyword is except. So, you are saying take this except what is here. So, these three set operations often give us very useful way of forming certain different types of queries.

(Refer Slide Time: 6:14)

Set Operations (2)

- Find the salaries of all instructors that are less than the largest salary
`select distinct T.salary
from instructor as T, instructor as S
where T.salary < S.salary`
- Find all the salaries of all instructors
`select distinct salary
from instructor`
- Find the largest salary of all instructors
`(select "second query")
except
(select "first query")`

Handwritten notes: A blue circle around `T.salary` in the first query, a blue circle around `S` in the first query, and the word "Salary" written vertically in blue ink next to the second query.

Set Operations (2)

- Find the salaries of all instructors that are less than the largest salary
`select distinct T.salary
from instructor as T, instructor as S
where T.salary < S.salary` ✓
- Find all the salaries of all instructors
`select distinct salary
from instructor`
- Find the largest salary of all instructors
`(select "second query")
except
(select "first query")`

Handwritten notes: Blue arrows pointing from the "second query" and "first query" in the third query to the `select distinct salary from instructor` query in the second query, and from the `select distinct salary from instructor` query to the `where T.salary < S.salary` condition in the first query.

Here is we show another example. We want to find out given the instructors; in the instructor table you remember that there is a salary field. We want to find out what is the largest salary, highest salary of all instructors. It is a different matter that we can use aggregate functions as we will see later on where it becomes very easy to just do things like highest or lowest or average, but let us say if we try to do it in the usual relational way how we can do it.

So, what is highest? Highest is difficult to define one value which is highest and it can be multiple actually, because multiple faculty may have the same highest salary. So, I can look at highest in a little different way, I can say that it may be easier to find instructors or find salaries which are not highest. So, when is a salary not highest? When it is less than some other salary, if a salary is less than some other salary, then it is not highest.

So, I first, this is my target, this is what I want to actually do, but this is what I start with. I find all instructors that salaries of all the instructors that are less than the largest salary, which I do not know. So, it is a naturally distinct t dot salary, so I have to do a comparison, so I have to do, in implicitly I have to do a join, I have to do a Cartesian product join, rather a Cartesian product not the natural join, it is a theta join.

So, I take the instructor as T as S and suppose I am taking out t dot salary in distinct terms. What is the condition? Condition is t dot salary is less than S dot salary. So, this will mean that only salary which will not satisfy this is the highest salary otherwise the highest salary will exist somewhere in s dot salary, so it will come in the combination everywhere with t dot salary and I will be able to get those t dot salary satisfying this condition and coming here except for those entries where the salary is the highest.

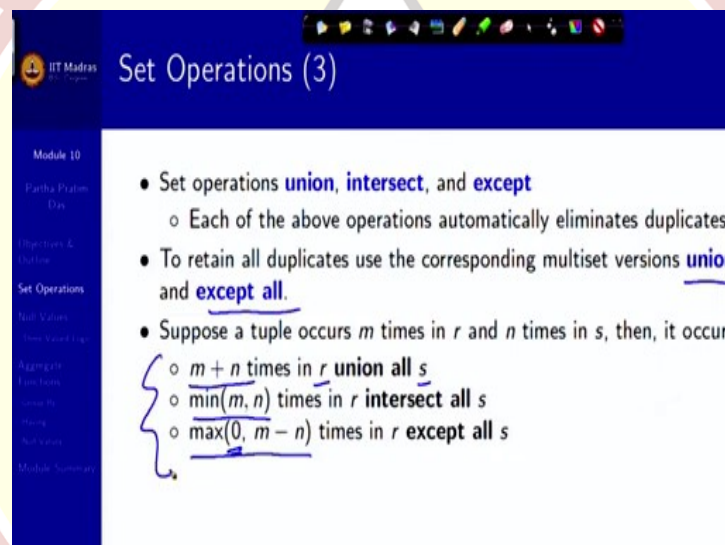
Now, once I have got that then the problem is simple, because I know the total set of salaries, I know the set of salaries that are not highest, so I will take out from the total set of salaries I will take out the salaries that are not highest, so what will remain is a highest salary. So, here I find all salaries which is very simple just doing a select distinct salary. Once this has been done, I will from this, this is what is referred to as a second query here.

From that I will do a difference by except for the first set, so this is set of all salaries, this is set of salaries that are not the highest, so if I do except what will remain is just the highest salary, I have already made things distinct here, so this will leave me with the table which has only one value, which is the highest salary.

And mind you, this is where it will be different from doing it using the aggregate function where at the end I do not get a table, I actually get the value. Here I will get the value as a part of the, as a record of single field in the table. So, this is how and set operations are come out to be very handy often because if you can think certain things in a relational way, traditional relational algebra, then it gets easier to directly put it to set operations.

Well not always it is the most efficient; that is a different thing. We will come to the efficiency and optimization and elegance of doing things later on, but just we are trying to look at the expressiveness, how easily we can express what we want to write as a query in terms of the SQL features, so this is one way of doing that.

(Refer Slide Time: 11:32)



Set Operations (3)

- Set operations **union**, **intersect**, and **except**
 - Each of the above operations automatically eliminates duplicates
- To retain all duplicates use the corresponding multiset versions **union all** and **except all**.
- Suppose a tuple occurs m times in r and n times in s , then, it occurs
 - $m + n$ times in r **union all** s
 - $\min(m, n)$ times in r **intersect all** s
 - $\max(0, m - n)$ times in r **except all** s

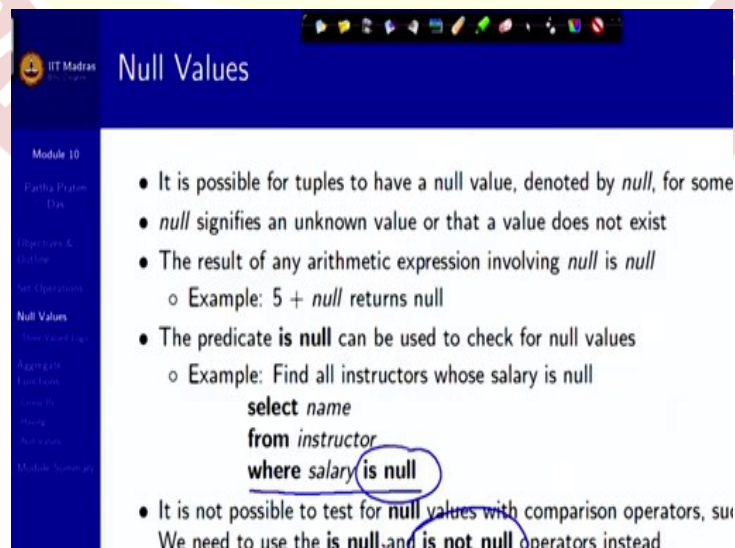
Now, certainly set operations as the name suggest is set, not multiset, so it treats every relation as a set which means that you do union intersection, intersect or except the result is unique, it is a no duplicate result unlike the normal relations in SQL. So, but in contrast if you want the duplicates to be retained, then you will have to use like this, union all, intersect all or except all, then all duplicates will also be retained.

So, that means if you use all with duplicates then what will happen if you are doing union of r and s where r has occurrence of a tuple m times and s has the occurrence of the tuple n times, then in the union it will have occurrence of m plus n times, because you are not eliminating anything, but if you do not do union all, then it will have only one.

Similarly for intersection, naturally it is a commonality, so what it will leave you is if r has occurrences of a tuple five times, s has occurrences of a tuple three times, only three are common considered common, so only three will come, so that is the minimum and in case of except naturally it is m minus n , so five are in r , three are in s , so as if three has been taken out so you are left with two.

But if you have the reverse, r has three and s has five, you cannot take out five from three, duplicate entries of a tuple, so your value is zero. So, it cannot be negative, the number of occurrences cannot be negative so it will simply not feature, so that is how this counts will have to be understood.

(Refer Slide Time: 13:44)



Next we go to null values because we have said what is the null value? Null value is unknown; I do not know what the value is. Why is null value important? For different reasons, one is certainly I may not, certain values may not exist for everyone as I said every student may not have a passport, so what value do I put.

That is one context which is very prevalent. It is not good to put, you will say why, I mean, why not I will put some pattern but that is not good because if you put a pattern then you will have to check if that particular pattern exist in the field whenever you want to access that particular value or do something with that particular value, which is nullable.

So, null is a special concept, a special entity which means purely, semantically means unknown, I do not know. The other context where null is very important is often times while we are creating a record or inserting a record, we may not know all the values of all the fields, when a student joins, the student does not have any credit assigned, so the total credit is null. You can say 0, but 0 and null are different.

0 will mean that the student has not taken a credit, null means the credit has not been given yet, it is unknown, so semantics are similar but different and significantly different, so that subsequently when the student takes credit, the credit will be put together. When we talked about altering tables, adding more attributes to a table.

Naturally when we add an attribute on all rows of that table, existing records of that table, the value of that particular field cannot be known, cannot be set, so it will have to be initialized as if as a null value and then subsequently you can do updates and fill up the right value and so on. Obviously there will have to be some value otherwise why are you adding that attribute. So, null values has a lot of consequence.

And therefore when we use the null value let us see what it signifies. So, it is possible for tuples to have null values signifies unknown value or a value that does not exist. Now, what if it participates in some operation, for example, if it participates in an arithmetic expression, the interpretation is simple, if a null participates in any arithmetic expression that result is null, irrespective of everything else in that. How do I check for null?

I cannot do equality null. Here if you think about say a language like C, C plus plus, they brought in the concept of null. C have the concept of null which was in a post thought, so what is said that, well, null is a defined value 0, so you could write it, you can write it as 0 if

pointer p is equal to is equal to 0 or you can write it as null, if you write it as null then people can read it better, that is it. So, even though the null exist in C, it is not a special value.

It is not a, I mean, I should say that it is also a valid value, it is treated as a special value. C plus plus, coming to C plus plus 11 did something better, they introduced null as a separate literal, as a special value though the semantics of zero continues. So, here in database you do not know what that null value is, so you cannot do a comparison with null. So, all that you can do is check if something is null or is not null, that is it.

But you cannot say equal to equal to null, if you do equal to equal to null the result is null, always false. So, if we say if the salary is not specified the way to check that will be salary is null. So, what will it do for a tuple, if the salary field is null it will return true otherwise it will return false, if you want to do the reverse you can do is not null or you can write the condition and negate it. Either way this will do, so this is the basic semantics and use of null.

(Refer Slide Time: 19:03)

Null Values (2): Three Valued Logic

- Three values – **true, false, unknown**
- Any comparison with *null* returns *unknown*
 - Example: $5 < \text{null}$ or $\text{null} <> \text{null}$ or $\text{null} = \text{null}$
- Three-valued logic using the value *unknown*:
 - OR: $(\text{unknown or true}) = \text{true}$,
 $(\text{unknown or false}) = \text{unknown}$
 $(\text{unknown or unknown}) = \text{unknown}$
 - AND: $(\text{true and unknown}) = \text{unknown}$,
 $(\text{false and unknown}) = \text{false}$,
 $(\text{unknown and unknown}) = \text{unknown}$
 - NOT: $(\text{not unknown}) = \text{unknown}$

Now, with null how does things work with the Boolean algebra? Boolean algebra has truth values, like true, false and now we are getting something which is unknown. Now what is unknown, is unknown true or is unknown false, is unknown something different, naturally unknown is neither true nor false, but it will feature with true and false more often.

So, what is done is the Boolean or two valued logic is extended to three values where you have the three literals, three constants, true, false and unknown. Now, and the way they interact between themselves in terms of the 'or', 'and' and 'not' the universally complete set

of operations is that between true and false, everything exist as in the Boolean algebra, nothing changes.

The definition of 'or', 'and' and 'not' for values true and false are the same as Boolean algebra. So, all that you need to see, need to learn and understand is how does it work when one or both of the operands actually is unknown. So, if you look at or, then there are three possibilities there, unknown or true, I mean assuming that unknown or true and true or unknown, associativity I am not considering because associativity still continues to hold.

It has no consequence, unknown or false and unknown or unknown. Now we say that unknown or true the result is taken to be true because or is either, so if I have two options, I know one is true and the other is not known what it is that is like doing a shortcut in Boolean evaluation, we often do that. If there are two conditions and if the first condition checked is turned out to be true, then we do not evaluate the second condition.

What is the meaning of not evaluating the second condition? I do not care whether it is true or false or said in other words that the other condition is actually unknown. So, this is true. But if it is unknown or false one side I know is false, it goes out and the other side I do not know, so I do not know, unknown or unknown has to be unknown, nothing is known.

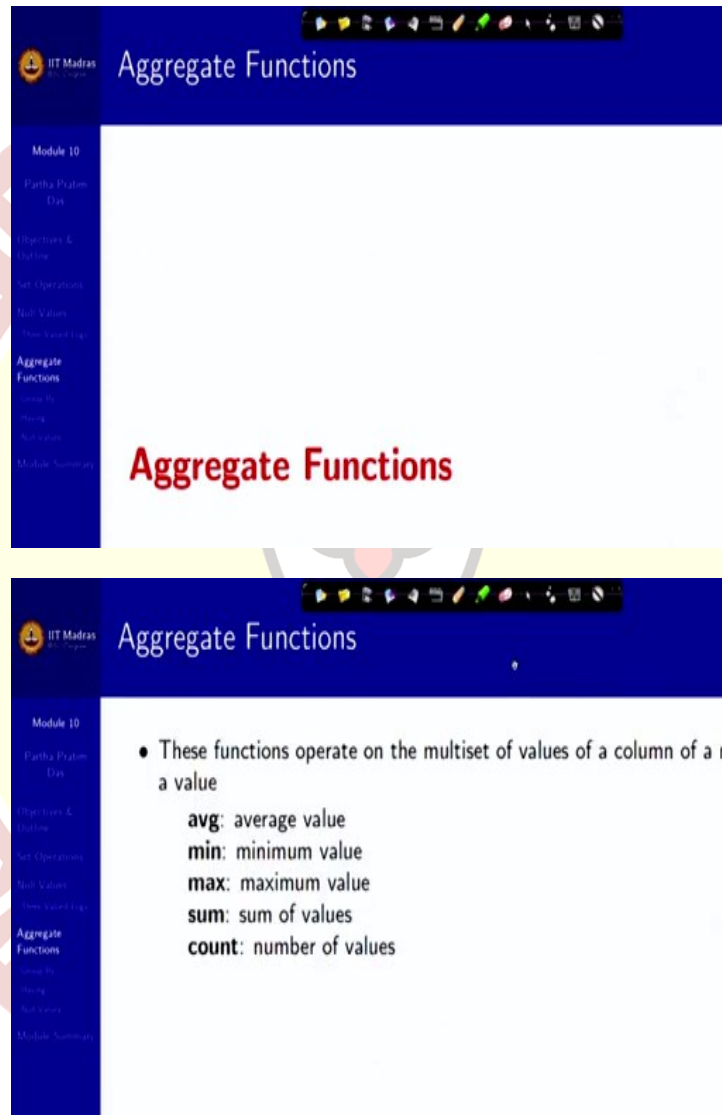
Similarly for and, true and unknown in a similar way because both will have to be true for truth to come, one is true, other is unknown, so it is unknown, but false and unknown is exactly, I mean, the other type of the shortcut, which says that false and unknown is false, I already know false, so it does not matter what, whatever value actually might be there, might have been there, whether it is true or false will make this false.

Unknown and unknown will have to be unknown; naturally if you do not know something then you do not know that you do not know something. So, not unknown is also unknown. Now, what happens is if p is unknown, then it evaluates to true if predicate p values to unknown that is null what we said. Now, when it occurs, I mean, finally you will have to take things to the Boolean algebra because that is where it will feature in the where clause.

So, the result of where clause predicate will be treated as false, treated as false, if it evaluates to unknown, if you get the result of the predicate through this three valued logic as unknown, then we will treat it as false and go ahead with the where clause, so that is the basic special interpretation to know.

Naturally any comparison that you do, any logical comparison that you do or rather relational comparison that you do between different values one of which is unknown or both of which is unknown has a result which is unknown. So, with this now it becomes clear as to, well defined, as to how we will use unknown in the overall scenario of writing where clauses.

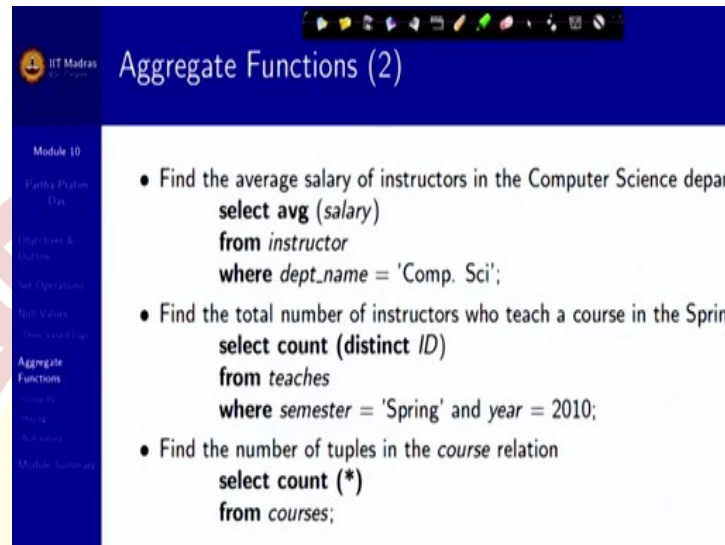
(Refer Slide Time: 23:52)



Let us, before I close today let me take, I am sorry a look at the aggregate functions, we talked about 5 aggregate functions, average, minimum, max and sum, their names will tell you, so this functions operate on the multiset of values of a column. Now naturally you are finding, these are all arithmetic functions, so you cannot do it on two columns or more columns. You have to choose a column on which you are doing this.

And naturally as you choose a column you have a multiset of values, there could be duplicates and it operates with those duplicates and what will it result, it will result in one single value.

(Refer Slide Time: 24:37)



Aggregate Functions (2)

- Find the average salary of instructors in the Computer Science department
`select avg (salary)
from instructor
where dept_name = 'Comp. Sci';`
- Find the total number of instructors who teach a course in the Spring semester
`select count (distinct ID)
from teaches
where semester = 'Spring' and year = 2010;`
- Find the number of tuples in the *course* relation
`select count (*)
from courses;`

So, like these are quite obvious, average salary of instructors, so instructor, in computer science department, so this is your where clause, the selection clause and the selection clause tells you that computer science faculty, choose the salary column and do the average. So, this will do the average. Now, this particularly becomes a problem when you take to count or maybe even in terms of average salary also will be biased by multiple.

But if you take a count often you would want, you may want the distinct, so if you want the distinct, then you know how to do it, instead of using ID use distinct ID, so the multiset will become set and then you count on that set. Prior to that this predicate tells you which tuples are coming in. Total number of tuples count star. What is star? It is a wild keyword, wild card, so any attribute, it does not matter which one.

That is just the semantic difference you will have to keep in mind is earlier when we did select star we meant all attributes, here when you do count star it actually means any attribute, because the count of records will be the same no matter which attribute you use.

(Refer Slide Time: 26:16)

Aggregate Functions (3): Group By

- Find the average salary of instructors in each department

```
select dept_name, avg(salary) as avg_salary
from instructor
group by dept_name;
```

ID	name	dept_name	salary
76766	Crick	Biology	72000
45565	Katz	Comp. Sci.	75000
10101	Srinivasan	Comp. Sci.	65000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000
12121	Wu	Finance	90000
76543	Singh	Finance	80000
32343	El Said	History	60000

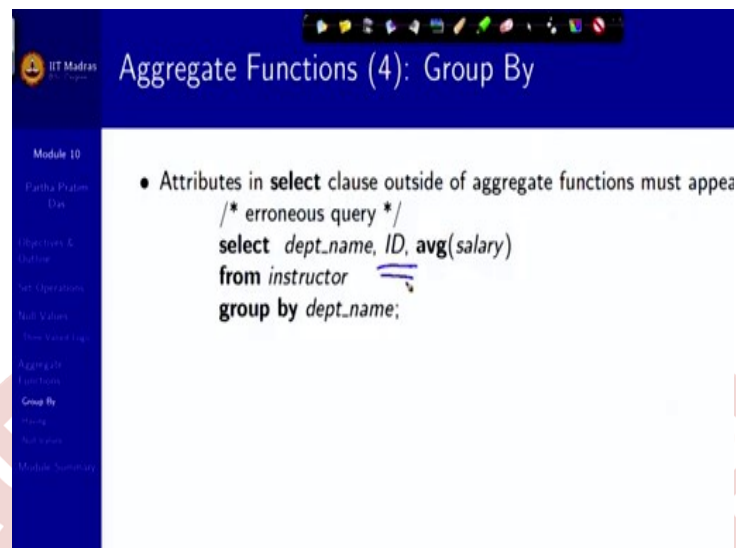
dept_name	avg.
Biology	72
Comp. Sci.	77
Elec. Eng.	80
Finance	85
History	61

So, these are some of the, this is an example, find average salary of instructors in each department. So, this is what we are trying to do. This is the entire instructor record, this is, these are different groups of instructors in different departments. Now, if I do average salary, department name average salary, then I will get the average of this entire thing. We do not want that. We want the average in every department.

So, what you want to do is combine these three together and find the average of these three values and put it here, combine these two together find the average of these two values and put it here; that can be done by group by. So group by takes a column and kind of makes sub tables you can think of. So, it makes sub tables of all those records which have the same value on that particular group by attribute.

And then applies the aggregate function on the column based on this sub table, so it is grouping this together and then finding the average and naturally in the result you need two columns, now one is what you have grouped on and the average value. So, you have the department name because you have grouped on that and you have the average salary which you have given at a different name that is decorative. So, group by is very-very convenient because often aggregation is not required globally, but is required based on certain attribute and you use group by on that.

(Refer Slide Time: 28:28)



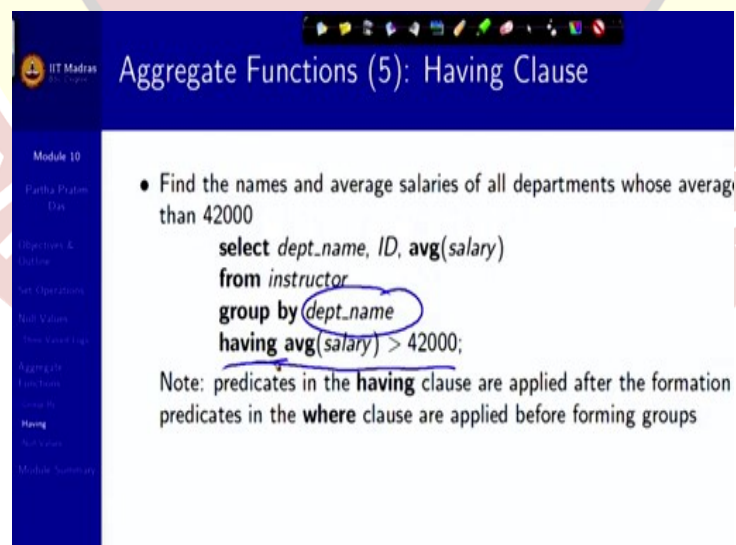
The slide is titled "Aggregate Functions (4): Group By". It contains a bullet point stating: "Attributes in **select** clause outside of aggregate functions must appear in the **group by** clause. /* erroneous query */". Below this, the following SQL query is shown:

```
select dept_name, ID, avg(salary)
from instructor
group by dept_name;
```

 A red arrow points to the `ID` attribute in the `select` clause, indicating it is not in the `group by` clause.

Now, remember that when you use group by the attribute in the select clause has to be limited only to the aggregation function and whatever you have group by, you can group by multiple, but only those attributes will have to come in the select, like in here ID has been put that is not right, this will not pass as a correct query.

(Refer Slide Time: 29:01)



The slide is titled "Aggregate Functions (5): Having Clause". It contains a bullet point stating: "Find the names and average salaries of all departments whose average salary is greater than 42000". Below this, the following SQL query is shown:

```
select dept_name, ID, avg(salary)
from instructor
group by dept_name
having avg(salary) > 42000;
```

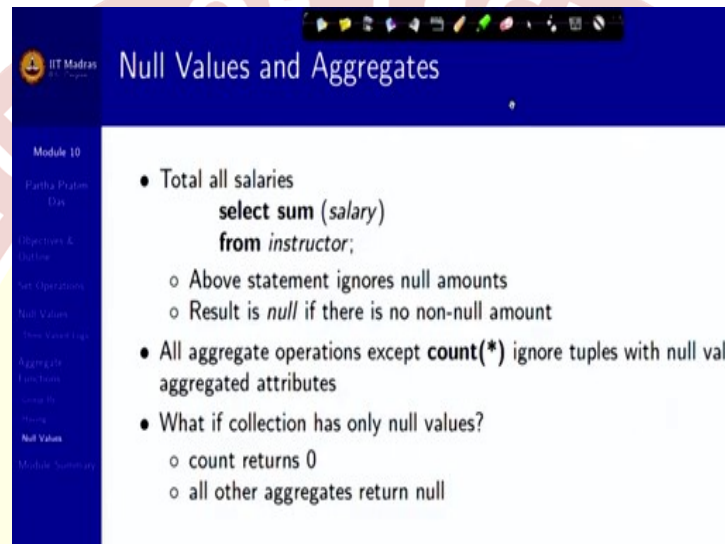
 A red circle highlights the `dept_name` attribute in the `group by` clause, and a red arrow points to the `avg(salary)` expression in the `having` clause. Below the query, a note states: "Note: predicates in the **having** clause are applied after the formation of groups. predicates in the **where** clause are applied before forming groups".

Finally, there are some refinements possible on the group by. What you can do is when you have grouped you can say that the value may have some properties and those properties are specified by having clause. So, I have seen certain kind of different predicates for where

clause which are based on individual records, but this is having clause has predicates which are applied after the formation of the group.

First the group is formed, department name, then on that group if you want to put some condition then you have the having clause, that is how it differs from the where clause which applies to before forming the group, so that is another convenient way of writing queries.

(Refer Slide Time: 29:54)



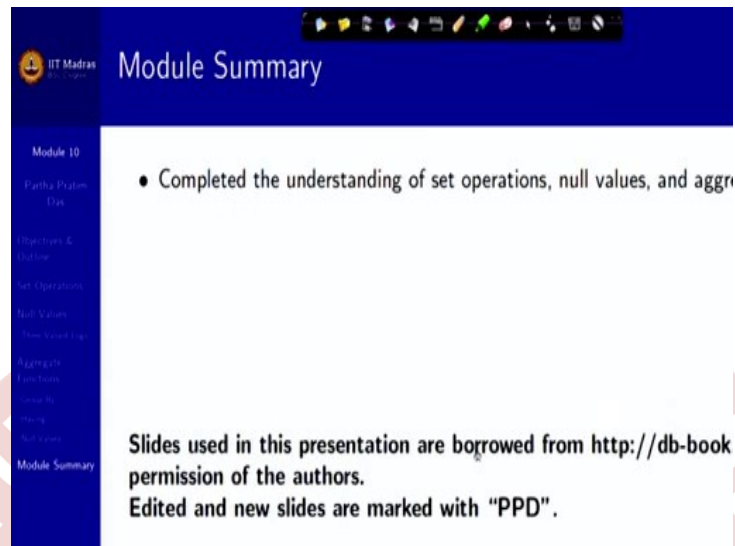
Null Values and Aggregates

- Total all salaries
select sum (salary)
from instructor;
 - Above statement ignores null amounts
 - Result is *null* if there is no non-null amount
- All aggregate operations except **count(*)** ignore tuples with null value on aggregated attributes
- What if collection has only null values?
 - count returns 0
 - all other aggregates return null

Finally, null values work in certain ways, for example, if you do a sum, then it will ignore the null values, if something is null value it will ignore and all aggregation operation other than count star will ignore null values on the aggregated attribute, because, I mean, the value is not known so and if it is a reality that you do not know the value you still have to get the result, so maybe the average will not include those records which has a null value on that field.

But if the entire collection has only null values, then if I do count it will give me a 0 because nothing is known, but if I do any other aggregation it will also return a null, even if there is one value it will ignore everything and give you, but if all values are null, then all aggregations other than count will give me null.

(Refer Slide Time: 31:13)



So, that is a certain level of coverage for the SQL language and features. There is lot more to do starting next week as well as some more examples to discuss. Try to understand these features as much as you can and work examples on the university database, but we will have more in the next week. Thank you all very much for the attention, see you in module 11 in week 3.